Reviewer Pack — Minimal Lattice Point Counting in Real Projective Space and …

Entry e19fa865152b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:00:35 UTC

Minimal Lattice Point Counting in Real Projective Space and Communication Complexity Rank Inequality

Entry ID: `e19fa865152b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 02:00:35 UTC

1. Conjecture

Field A (mathematical branch): Discrete Geometry (Lattice Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For any given n-vertex graph G, the minimal number of lattice points in the real projective space spanned by the vertices of G, denoted as L(G), is lower-bounded by the rank of the communication matrix of G, such that L(G) ≥ c * r(G), where r(G) is the rank of G and c is a constant.

Rationale (proposer's reasoning):

> Lattice theory provides a geometric structure that can capture the connectivity of a graph. The communication complexity of a graph measures the minimum number of bits required to communicate information between nodes in the worst case. This conjecture suggests that there is a direct relationship between the geometric properties of a graph and its communication complexity, which could provide new insights into understanding the fundamental limits of communication.

Taxonomy category: `lattice_theory_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `de2106ccac22bb5f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across at least 24 out of 30 seeds, the ratio between the minimal lattice point count L(G) and the rank r(G) of the communication matrix for each graph G is greater than or equal to c, where c is a pre-specified constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_random_graph(n, max_degree):
    graph = [[] for _ in range(n)]
    degree = [0] * n
    edges_added = 0

    while edges_added < n * max_degree // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        if u != v and v not in graph[u]:
            graph[u].append(v)
            graph[v].append(u)
            degree[u] += 1
            degree[v] += 1
            edges_added += 1

    return graph

def compute_communication_matrix(graph):
    n = len(graph)
    comm_matrix = [[0] * n for _ in range(n)]

    for u in range(n):
        for v in range(u + 1, n):
            if v in graph[u]:
                comm_matrix[u][v] = 1
                comm_matrix[v][u] = 1

    return comm_matrix
```

```
def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0

    for i in range(n):
        pivot_row = next((j for j in range(rank, m) if matrix[j][i] != 0), None)
        if pivot_row is None:
            continue

        # Swap rows
        matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]

        # Make the pivot element 1
        denom = matrix[rank][i]
        for j in range(n):
            matrix[rank][j] /= denom

        # Eliminate other elements in the column
        for j in range(m):
            if j != rank:
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[rank][k]

        rank += 1

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    n_max = 0

    for n in n_values:
        graph = generate_random_graph(n, max_degree=3)
        comm_matrix = compute_communication_matrix(graph)

        rank = gaussian_elimination(comm_matrix)
        lattice_points = len([x for x in range(1 << n) if all((x >> i) & 1 == (y >> i) & 1 or y no

        instances_tested += n
        n_max = max(n_max, n)

        total_metric_value += lattice_points / rank

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = all(lattice_points >= c * rank for lattice_points, rank in zip(lattice_poin

    return {
        "metric_name": "L(G) / r(G)",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
```

```
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='not supported' first_failing_seed={first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83cd4790.py", line 113, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83cd4790.py", line 88, in run_trial
    lattice_points = len([x for x in range(1 << n) if all((x >> i) & 1 == (y >> i) & 1 or y not in graph[i]
                                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_83cd4790.py", line 88, in <genexpr>
    lattice_points = len([x for x in range(1 << n) if all((x >> i) & 1 == (y >> i) & 1 or y not in graph[i]
                                                                                    ~~^^~~
TypeError: unsupported operand type(s) for >>: 'list' and 'int'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required number of trials to meet the pre-registered support condition. | next: Re-run the test with proper error handling and ensure that it completes all necessary trials to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15785
2	propose	ollama_remote	glm4:latest	0	0	12394
3	propose	ollama_remote	glm4:latest	0	0	12820
4	propose	ollama_remote	glm4:latest	0	0	13723
5	preregistration	ollama_remote	glm4:latest	0	0	9835
6	novelty	ollama_remote	glm4:latest	0	0	13186
7	novelty	ollama_remote	glm4:latest	0	0	8696
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20645
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12519
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13487
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11762
12	judge	ollama_remote	glm4:latest	0	0	15953

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 160807 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/e19fa865152b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e19fa865152b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e19fa865152b.tar.gz` (if generated)